Deep Learning

Assignment 10

Using LSTM for prediction of future weather of cities in Python

Name: Rohit Kalaskar
Roll No: 24
PRN: 12310106
Branch: CS AI - B

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

```
data = pd.read_csv("weather_data.csv")
city_name = "San Diego"
data = data[data['Location'] == city_name].copy()
data['Date_Time'] = pd.to_datetime(data['Date_Time'])
data = data.sort_values('Date_Time')
data.set_index('Date_Time', inplace=True)
data = data[['Temperature_C', 'Humidity_pct', 'Precipitation_mm', 'Wind_Speed_kmh']]
data = data.ffill()
data['month'] = data.index.month
data['day'] = data.index.day
print(f"Dataset shape for {city_name}:", data.shape)
data.head()
```

```
Dataset shape for San Diego: (6805, 6)
```

Date_Time	Temperature_C	Humidity_pct	Precipitation_mm	Wind_Speed_kmh	month	day
2024-01-01 00:16:59	-7.321730	75.600117	1.460072	15.595410	1	1
2024-01-01 00:18:36	-1.339214	64.766453	4.767101	12.546028	1	1
2024-01-01 00:37:46	34.779935	82.665749	2.551479	21.042542	1	1
2024-01-01 00:39:40	8.077836	77.265854	0.293375	5.117413	1	1
2024-01-01 00:56:04	-0.157275	35.239467	8.167008	15.575991	1	1

```
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)
def create_dataset(data, time_step=30):
    X, Y = [], []
    for i in range(len(data) - time_step - 1):
        X.append(data[i:(i + time_step)])
        Y.append(data[i + time_step, 0])
    return np.array(X), np.array(Y)
time_step = 30
X, y = create_dataset(scaled_data, time_step)
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
print("X shape:", X.shape)
print("X_train:", X_train.shape)
print("X_test: ", X_test.shape)
```

```
X shape: (6774, 30, 6)
X_train: (5419, 30, 6)
X_test:  (1355, 30, 6)
```

```
model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(time_step, X.shape[2])),
    Dropout(0.2),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(32, activation='relu'),
```

```
    Dense(1)
])
model.compile(loss='mean_squared_error', optimizer='adam')
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_di
  super().__init__(**kwargs)
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
lstm_10 (LSTM)	(None, 30, 128)	69,120
dropout (Dropout)	(None, 30, 128)	0
lstm_11 (LSTM)	(None, 64)	49,408
dropout_1 (Dropout)	(None, 64)	0
dense_6 (Dense)	(None, 32)	2,080
dense_7 (Dense)	(None, 1)	33

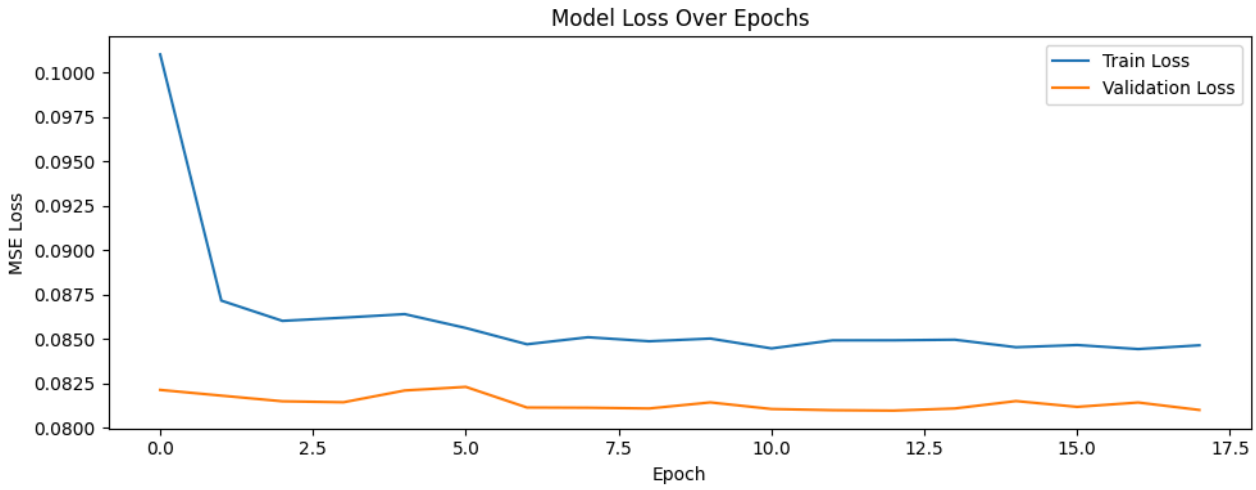
Total params: 120,641 (471.25 KB)
Trainable params: 120,641 (471.25 KB)
Non-trainable params: 0 (0.00 B)

```
callbacks = [
    EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=3,
        verbose=1
    )
]
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=64,
    validation_split=0.1,
    callbacks=callbacks,
    verbose=1
)
```

```
Epoch 1/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 6s 52ms/step - loss: 0.1010 - val_loss: 0.0821 - learning_rate: 0.0010
Epoch 2/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 5s 50ms/step - loss: 0.0872 - val_loss: 0.0818 - learning_rate: 0.0010
Epoch 3/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 49ms/step - loss: 0.0860 - val_loss: 0.0815 - learning_rate: 0.0010
Epoch 4/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 58ms/step - loss: 0.0862 - val_loss: 0.0814 - learning_rate: 0.0010
Epoch 5/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 51ms/step - loss: 0.0864 - val_loss: 0.0821 - learning_rate: 0.0010
Epoch 6/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0856
Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 49ms/step - loss: 0.0856 - val_loss: 0.0823 - learning_rate: 0.0010
Epoch 7/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 6s 57ms/step - loss: 0.0847 - val_loss: 0.0811 - learning_rate: 5.0000e-04
Epoch 8/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 49ms/step - loss: 0.0851 - val_loss: 0.0811 - learning_rate: 5.0000e-04
Epoch 9/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 50ms/step - loss: 0.0849 - val_loss: 0.0811 - learning_rate: 5.0000e-04
Epoch 10/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step - loss: 0.0854
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 55ms/step - loss: 0.0850 - val_loss: 0.0814 - learning_rate: 5.0000e-04
Epoch 11/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 5s 49ms/step - loss: 0.0845 - val_loss: 0.0811 - learning_rate: 2.5000e-04
Epoch 12/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 56ms/step - loss: 0.0849 - val_loss: 0.0810 - learning_rate: 2.5000e-04
Epoch 13/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 49ms/step - loss: 0.0849 - val_loss: 0.0810 - learning_rate: 2.5000e-04
Epoch 14/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 49ms/step - loss: 0.0850 - val_loss: 0.0811 - learning_rate: 2.5000e-04
Epoch 15/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - loss: 0.0849
Epoch 15: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 57ms/step - loss: 0.0845 - val_loss: 0.0815 - learning_rate: 2.5000e-04
Epoch 16/50
```

```
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 49ms/step - loss: 0.0847 - val_loss: 0.0812 - learning_rate: 1.2500e-04
Epoch 17/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 56ms/step - loss: 0.0844 - val_loss: 0.0814 - learning_rate: 1.2500e-04
Epoch 18/50
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - loss: 0.0859
Epoch 18: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
77/77 ━━━━━━━━━━━━━━━━━━━━ 4s 57ms/step - loss: 0.0846 - val_loss: 0.0810 - learning_rate: 1.2500e-04
Epoch 18: early stopping
Restoring model weights from the end of the best epoch: 13.
```

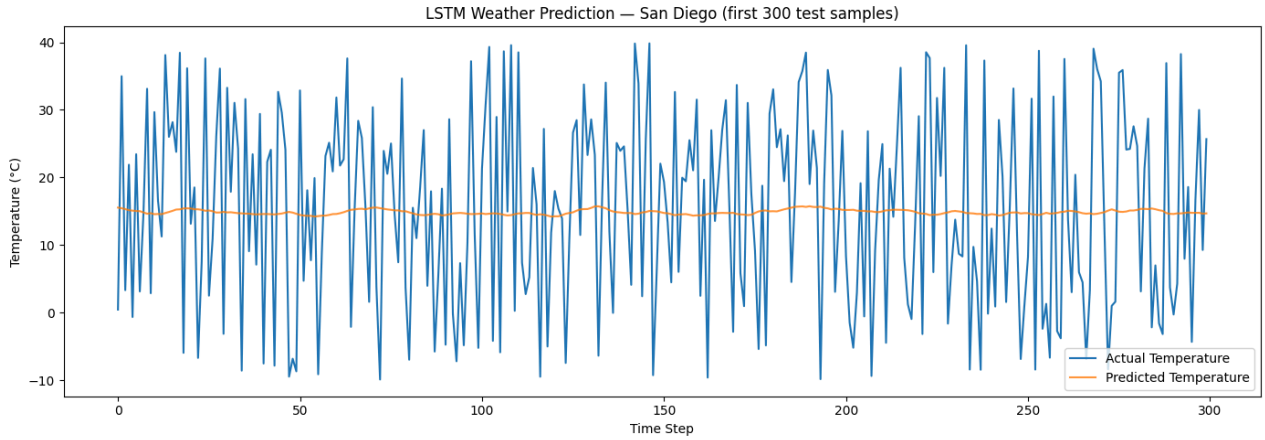
```
plt.figure(figsize=(10, 4))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.legend()
plt.tight_layout()
plt.show()
```




```
train_predict = model.predict(X_train)
test_predict  = model.predict(X_test)
def inverse_temp(pred):
    temp = np.zeros((len(pred), scaled_data.shape[1]))
    temp[:, 0] = pred[:, 0]
    return scaler.inverse_transform(temp)[:, 0]
train_predict    = inverse_temp(train_predict)
test_predict     = inverse_temp(test_predict)
y_train_actual   = inverse_temp(y_train.reshape(-1, 1))
y_test_actual    = inverse_temp(y_test.reshape(-1, 1))
train_rmse = np.sqrt(mean_squared_error(y_train_actual, train_predict))
test_rmse  = np.sqrt(mean_squared_error(y_test_actual,  test_predict))
print(f"Train RMSE: {train_rmse:.4f} °C")
print(f"Test  RMSE: {test_rmse:.4f} °C")
```

```
170/170 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step
43/43 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step
Train RMSE: 14.4526 °C
Test  RMSE: 14.3727 °C
```

```
plt.figure(figsize=(14, 5))
plt.plot(y_test_actual[:300],  label='Actual Temperature',    linewidth=1.5)
plt.plot(test_predict[:300],   label='Predicted Temperature', linewidth=1.5, alpha=0.8)
plt.title(f"LSTM Weather Prediction — {city_name} (first 300 test samples)")
plt.xlabel("Time Step")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.tight_layout()
plt.show()
```




```
future_steps   = 7
last_sequence  = scaled_data[-time_step:].copy()
future_predictions = []
for step in range(future_steps):
    pred = model.predict(last_sequence.reshape(1, time_step, X.shape[2]), verbose=0)
    future_predictions.append(pred[0][0])
    new_row = last_sequence[-1].copy()
    new_row[0] = pred[0][0]
    last_date = data.index[-1] + pd.Timedelta(days=step + 1)
    temp_row = np.zeros(scaled_data.shape[1])
    temp_row[0] = new_row[0]
    temp_row[1] = new_row[1]
    temp_row[2] = new_row[2]
    temp_row[3] = new_row[3]
    temp_row[4] = (last_date.month - 1) / 11.0
    temp_row[5] = (last_date.day   - 1) / 30.0
    new_row = temp_row
    last_sequence = np.vstack((last_sequence[1:], new_row))
future_predictions = inverse_temp(np.array(future_predictions).reshape(-1, 1))
last_date = data.index[-1]
future_dates = [last_date + pd.Timedelta(days=i+1) for i in range(future_steps)]
print("\nNext 7 Days Temperature Forecast:")
for d, t in zip(future_dates, future_predictions):
    print(f"  {d.date()} → {t:.2f} °C")
plt.figure(figsize=(10, 4))
plt.plot(future_dates, future_predictions, marker='o', linewidth=2, color='tomato', label='Forecast')
plt.title(f"7-Day Temperature Forecast — {city_name}")
plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.xticks(rotation=30)
plt.legend()
plt.tight_layout()
plt.show()
```

```
Next 7 Days Temperature Forecast:
  2024-05-19 → 14.99 °C
  2024-05-20 → 14.97 °C
  2024-05-21 → 14.99 °C
  2024-05-22 → 15.05 °C
  2024-05-23 → 15.15 °C
  2024-05-24 → 15.26 °C
  2024-05-25 → 15.39 °C
```